

SOFTWARE REQUIREMENTS SPECIFICATION

Enterprise Human Resource Management (HRM) System
& ZKTeco Biometric Device Integration

Document Version: 1.0.0

Date: June 2, 2026

Status: Approved / Production-Ready Specification

1. Introduction

1.1 Purpose

This document details the functional, non-functional, and integration requirements for the Enterprise Human Resource Management (HRM) System. It provides clear engineering instructions for developers, testers, and infrastructure specialists implementing the platform core modules and connecting to **ZKTeco M2 Series** biometric devices.

1.2 Scope

The solution covers five primary domains—Calendar, Employee Profiles, Attendance Tracking, Leave Automation, and Payroll Processing—alongside an integrated low-level communication gateway interfacing directly with physical Linux-based ZKTeco M2 attendance devices.

2. Module Functional Requirements

2.1 Calendar Module

The baseline context system responsible for mapping time constraints over organizational hierarchies.

- **Company Holiday Registry:** Administrators must be able to define statutory, regional, floating, and localized company holidays. These records must automatically flag operational dependencies to dynamically block routine absenteeism logic.
- **Operational & Shift Events:** Capabilities to overlay enterprise events directly onto specific departmental sub-calendars, affecting baseline shift obligations without deleting permanent scheduling configurations.

2.2 Employees Module

The centralized Single Source of Truth (SSOT) managing operational identity matrices.

- **Profile Ledger:** Complete tracking of personal profile properties, emergency datasets, regulatory configurations, active/historical job titles, reporting matrices, and validated bank/tax attributes.
- **Biometric Endpoint ID Association:** Each profile schema includes a strict, unique alphanumeric field mapping directly to the physical `User_ID` configured inside the ZKTeco hardware ecosystem.

2.3 Attendance Module

The high-throughput component digesting punch streams, enforcing administrative frameworks, and generating structured timesheets.

Core Sub-Component Definitions:

- **Attendance Record:** The system logs all absolute entries containing a unique ID, explicit timestamp data, direction parameters (In/Out), and hardware-source signatures.
- **Timesheet System:** An analytical calculation framework summarizing daily metrics into monthly/cycle views showing productive hours, break intervals, and structural anomalies.
- **Attendance Regulation:** Rule system defining Grace Periods (allowable late intervals), early checkout thresholds, and minimum block bounds to qualify for partial presence credits.
- **Shifts Configuration:** Supports multiple variants including standard fixed durations, flexible/rolling thresholds, and sequential multi-week team rotation matrices.
- **Attendance Policies:** Operational rules that convert compliance anomalies (e.g., three late arrivals within a rolling monthly window) into specific disciplinary salary deductions.

2.4 Leave Management Module

The administrative subsystem managing employee time-off processes.

- **Leave Application Workflow:** Multi-tiered self-service approval request platform that automatically enforces rule validity blocks prior to supervisor review routing.
- **Leave Balances & Accrual Engine:** Processes ongoing monthly logic or upfront allocations, while respecting explicit year-end carry-over ceilings or cash-out payouts.
- **Leave Type Definitions:** System schema supporting Annual, Sick, Casual, Maternity, Paternity, and Unpaid Leave definitions, each bound to specific programmatic validation rules.

2.5 Payroll Management Module

The ultimate downstream consumer processing variable inputs into structural payments.

- **Employee Salaries:** Holds structural calculation formulas covering Fixed Earnings (Base, Housing, Travel allowances) and Statutory Deductions (Provident Fund, Tax models, Insurance schemes).
- **Payroll Runs:** A transactional engine that aggregates approved monthly timesheets, subtracts calculated unpaid absences, adds approved bonuses, and runs financial calculation passes.
- **Pay-slips Generation:** Creates cryptographic, tamper-proof, multi-page PDF documents outlining calculation lines for distribution via self-service modules.

3. ZKTeco M2 Hardware Integration Architecture

The ZKTeco M2 and M2F series terminals operate an embedded Linux operating system. Raw transactions are processed over local networks via TCP/IP on port 4370, or pushed over public networks using the proprietary HTTP ADMS protocol.

3.1 Dynamic Punch Processing Architecture

The system converts raw machine data points into analytical clock-in/out windows based on standard shift configurations. The logic maps entries using the following mathematical parsing framework:

Let raw biometric punches for an individual employee on date D be an ordered set: $P = \{p_1, p_2, \dots, p_n\}$ sorted chronologically.

Let the defined shift schedule start at time S_{start} and end at S_{end} , with the exact structural midpoint defined as: $M = S_{start} + \frac{1}{2}(S_{end} - S_{start})$

The automated sorting system maps parameters as follows:

- **Final Clock-In Allocation:** $Final_Clock_In = \min(\{p \in P \mid p \leq M\})$. If no such record exists, the system flags a missing clock-in event.
- **Final Clock-Out Allocation:** $Final_Clock_Out = \max(\{p \in P \mid p > M\})$. If no record matches this window, the system flags a missing departure event.

3.2 Complete Integration Middleware Core (Node.js Implementation)

This integration module handles persistent hardware socket listeners, extracts transaction arrays, translates internal verification bytes, and maps payloads directly to the core API layer.

```
/**
 * ZKTeco M2 Core System Integration Gateway Client
 * Enforces strict error mitigation, reconnection, and mapping configurations.
 */
const ZKClient = require('zk-attendance-sdk');

const TERMINAL_IP = '192.168.1.150';
const TERMINAL_PORT = 4370;
const CORE_HRM_INGEST_URL = 'https://api.hrm-system.internal/v1/attendance/raw-punch';

class HardwareIntegrationService {
  constructor(ip, port) {
    this.ip = ip;
    this.port = port;
    this.terminalInstance = null;
    this.isProcessing = false;
  }

  async bootstrapPipeline() {
    try {
      console.log(`[INIT] Opening Socket to ZKTeco M2 at: ${this.ip}:${this.port}`);
      this.terminalInstance = new ZKClient(this.ip, this.port);
      await this.terminalInstance.createSocket();
      console.log('--- CONNECTION ESTABLISHED SUCCESSFULLY ---');

      // Sync terminal clock to maintain system synchronization
    }
  }
}
```

```

        await this.terminalInstance.setTime(new Date());

        // Execute recurring retrieval sequence
        this.executeIngestionCycle();
    } catch (err) {
        console.error(`[CRITICAL] Connection Refused: ${err.message}`);
        this.retryAfterDelay();
    }
}

async executeIngestionCycle() {
    if (this.isProcessing) return;
    this.isProcessing = true;

    try {
        console.log('[DATA] Executing raw transaction log pull sequence...');
        const records = await this.terminalInstance.getAttendances();

        if (records && records.data && records.data.length > 0) {
            console.log(`[DATA] Discovered ${records.data.length} pending entries.`);
            for (const row of records.data) {
                await this.transformAndForward(row);
            }
        } else {
            console.log('[DATA] Zero unhandled transaction lines present on hardware.');
```

```

        4: 'RFID_PROXIMITY_CARD',
        15: 'FACE_RECOGNITION_BIOMETRIC'
    };
    return lookup[typeByte] || 'UNKNOWN_HARDWARE_INPUT';
}

retryAfterDelay() {
    console.log('[RETRY] Initiating link retry loop in 30 seconds...');
    setTimeout(() => this.bootstrapPipeline(), 30000);
}
}

// System execution block
const integrationWorker = new HardwareIntegrationService(TERMINAL_IP, TERMINAL_PORT);
integrationWorker.bootstrapPipeline();

```

4. Non-Functional & Compliance Requirements

4.1 Performance & Scalability

The database ingestion engine must support simultaneous log processing streams from up to 100 parallel ZKTeco devices without causing locking issues in the downstream Attendance_Record or Timesheet tables.

4.2 Security & Audit Logs

All network traffic pushing raw attendance telemetry from local gateways to cloud services must use TLS 1.3 encryption. Biometric templates stored on the devices must remain salted and hashed using cryptographic encryption standards to prevent reverse-engineering of physical user identities.

4.3 System Resilience

The integration middleware must preserve a local SQLite caching register to track processed transactions. If WAN connectivity drops, the middleware will automatically backfill missing records to the cloud endpoints using a chronological synchronization sequence once connection is restored.